

Introduction to R

AUTHOR

Denis Rubin, et al.

NYU Data Services: data.services@nyu.edu

PUBLISHED

August 24, 2026

I. Overview of R : R's Interface

The RStudio Interface: Four Main Panes

- 1. Source / Script Editor (Top Left)** This is where we write, edit, and save our code. Always write your code here so you have a reproducible record of your work! (You can put your cursor on a line and press **Ctrl + Enter** or **Cmd + Return** to run it).
- 2. Console (Bottom Left)** This is where R actually executes your code and prints the text output. You can type quick commands directly in here, but the code will not be saved when you close the program.
- 3. Environment (Top Right)** R only knows what it has actively memorized. This pane shows every dataset, variable, and value you have loaded. If you don't see your data listed here, R doesn't know it exists.
- 4. Files, Plots, & Help (Bottom Right)** This is where your visualizations will appear (Plots), where you can view your computer's folder structure (Files), and where official R documentation will open (Help).

Setting Up Your Workspace: RStudio Projects

Before we write a single line of code, we need to organize our digital workspace.

Why We Use RStudio Projects

- **Automatic Working Directory:** Sets the folder as your default directory, allowing you to load data without specifying file paths and working directories
- **State Restoration:** Instantly reopens the exact scripts and data files you were using when you last closed the program.
- **Cursor Placement:** Remembers exactly which line of code your cursor was on so you can pick up right where you left off.
- **Isolated Environments:** Keeps your command history and variables strictly separate from other assignments.
- **Ultimate Portability:** Uses relative file paths, meaning you can zip the folder, send it to a collaborator, and the code will run perfectly on their machine.

Quick Project Setup

1. Go to your computer's Desktop and create a brand new folder called `Intro_to_R`. Everything we do in this class will live in this folder.
2. If you haven't already, download this exact `.qmd` file we are looking at and physically drag it from your `Downloads` folder into your new `Intro_to_R` folder.
3. In RStudio, look at the very top-right corner. Click the light blue cube icon that says **Project: (None)**. * Select **New Project...** * Select **Existing Directory** * Browse to your `Intro_to_R` folder on your Desktop and click

Create Project.

4. Look at the **Files** tab in your bottom-right pane. You should now see this `.qmd` file sitting right there.

II. R Basics

There are several ways to execute code in R.

- Console execution: Type code in the Console window and press `Enter` to run it immediately.
- Line or selection: Press `Ctrl + Enter` (or `Cmd + Enter` on Mac) to run the current line or highlighted text.
- Code chunk: Press `Ctrl + Shift + Enter` (or `Cmd + Shift + Enter` on Mac) to run an entire script or Quarto/R Markdown chunk.

This is a code chunk:

```
1 # Recognizes 1 as a vector with only a single element
1 + 1
a
```

The `#` character within a code block creates a comment: text within your code that R will ignore/skip over.

A comment is human-readable explanatory text embedded directly inside source code.

Comments are written to explain why code was written a certain way, how a complex formula works, or what a specific section of code is meant to do. They are for humans (you and your collaborators) to read, not for the computer to execute

Assigning values

```
add <- 1 + 2
add # By just calling the name, you can see the value
```

Objects: `<object name> <- <information in R>`

```
value1 <- 1 + 2
value1

value2 <- "blue"
value2
```

Using Functions

```
sqrt(2 ^ 3)
sqrt(value1)
sqrt(value2)
```

```
help(sqrt)
?help
```

Data Types

1. Vectors

```
vec1 <- c(4.6, 4.8, 5.2, 6.3, 6.8, 7.1, 7.2, 7.4, 7.5, 8.6)
vec2 <- c('UT', 'IA', 'MN', 'AL', 'WI', 'MN', 'OH', 'IA', 'NY', 'IA')
?c # Combines values into a vector or a list

# Element of Vectors : <vector>[<vector of indices>]
vec1[7]
vec2[1:3]
vec2[c(1,3)]

# Length of the Vectors
length(vec1)
length(vec2)

# Class of Vectors
class(vec1)
class(vec2)

# Functions for a Numeric Vector
mean(vec1)
var(vec1)
sum(vec1)
max(vec1)

# Summaries of Vectors
summary(vec1)
summary(vec2)
summary(as.factor(vec2)) # If you classify obj2 as a factor variable, you can obtain frequencies
?as.factor()

as.character(vec1)
as.factor(vec1)

# Vector Operations
vec1 + 1
vec1 * 5
rep(1, times = 10)
rep(vec1, times = 5)
round(vec1 / 3, 2)
floor(vec1)

# Combine Two Vectors into a Single Vector with Length = 20
vec3 <- c(vec1, vec2)
length(vec3)
```

Exercise 1

Create a vector called `e1` which is a sequence of 1 to 10, repeating 5 times.

2. Data Frame

```
df1 <- data.frame(vec1, vec2)
class(df1)
str(df1) # compare with Global environment in RStudio

# Referencing via brackets : <dataframe>[<row indices>, <column indices>]
df1[, 1]
df1[, 2]

df1[, 'vec1']

# Referencing columns (variables) via names
df1$vec1
df1$vec2
```

Exercise 2

Create a data frame called `e2`, which adds a third variable (`idnum`, see below) to `df1`.

```
idnum <- 1:10
```

Missing Data

```
vec4 <- c(4.6, 4.8, 5.2, 6.3, 6.8, 7.1, 7.2, 7.4, 7.5, 8.6, NA)
sum(vec4) # Why can't the sum be calculated?
is.na(vec4)
sum(vec4, na.rm = TRUE)
is.na(vec4) # NA is still in the vector & only removed from the calculation
```

III. Getting Started in R

Packages

```
library() # Check all packages installed
search() # Check packages currently loaded
# You can also see this in the Packages tab in the bottom right pane.
```

To install a new package

1. `install.packages("name of package")`
2. `library(name of package)`

Note the presence and absence of quotes in these steps.

Can also use the drop down menu: Tools -> Install Packages

```
# Run following line in terminal - to install NHANES package
install.packages("NHANES")
```

NHANES is survey data collected by the US National Center for Health Statistics (NCHS). NHANES contains 75 variables available for the 2009-2010 and 2011-2012 sample years and 10,000 rows of data resampled from raw data. NHANES can be treated, for educational purposes, as if it were a simple random sample from the American population.

[NHANES Package Webpage](#)

```
# Load NHANES package - this loads in the data as well
library(NHANES)
```

Anybody can write a package if they want to, so there are some pretty funky packages out there!

```
# install.packages("cowsay")
library(cowsay)
cow <- animal_fetch('cow')
cat(cow)
```

```
  ^  ^
(oo)\_____/
(____)\       )\/\
    ||----w |
    ||     ||
```

```
pig <- animal_fetch("pig")
cat(pig)
```

```
 _//| .-~~~-.
_/_oo }      }-@
('')_ }      |
`--'| { }--{ }
    //_/ /_/ [nosig]
```

More info about cowsay: <https://cran.r-project.org/web/packages/cowsay/vignettes/cowsay.html>

There are many packages in CRAN (The Comprehensive R Archive Network).

There are over 24,103 available packages as of June 2026 (<https://cran.r-project.org/index.html>).

For dealing with data, choose ones that belong to the **tidyverse** (<https://tidyverse.org/packages/>):

- ***Vetted, commonly used, well documented***
- ***All packages share an underlying design philosophy, grammar, and data structures.***

```
install.packages("tidyverse")
library(tidyverse)
```

Importing Datasets

```
# 1. View data that has been loaded in from package
# (in this case the name of the package - this is not always the case).
View(NHANES)

# If the dataset is too large, you can look only at the beginning or end of the dataset specifically.
View(head(NHANES))

# or
View(tail(NHANES))

# For example, if you want to work with a subset of the data:
sample_nhanes <- head(NHANES, n = 10) # just the first 10 rows
```

Reading and Writing Data

Text Files (CSV, TXT, and so on)

You can use built-in functions:

```
write.csv(sample_nhanes, "sample_1.csv", row.names = FALSE)
sample_1 <- read.csv("sample_1.csv")
```

There are convenient packages that optimize reading and writing text files. Use the `readr` package, part of the tidyverse:

```
# install.packages('readr')
library(readr)
write_csv(sample_nhanes, 'sample_2.csv')
sample_2 <- read_csv('sample_2.csv')
```

For EXCEL files (XLS, XLSX), use the `readxl` package for reading and `writexl` for writing:

```
# install.packages("writexl")
library(writexl)
write_xlsx(sample_nhanes, path = "sample_3.xlsx")

# install.packages("readxl")
library(readxl)
sample_3 <- read_xlsx("sample_3.xlsx")
```

Read and write SAS, SPSS, and Stata Formats using the `haven` package:

```
# install.packages("haven")
library(haven)
write_sav(sample_nhanes, "sample_4.sav")
```

```
write_dta(sample_nhanes, "sample_5.dta")
write_xpt(sample_nhanes, "sample_6.xpt")

sample_4 <- read_sav("sample_4.sav")
sample_5 <- read_dta("sample_5.dta")
sample_6 <- read_xpt("sample_6.xpt")
```

IV. Piping in R

The Magic of the Pipe

If there is one concept that defines modern R programming, it is the pipe.

The pipe is a connective operator that passes the result on its **left** into the function on its **right**. Whenever you see a pipe in R, read it out loud as “...and then...”

The Rationale: The “Morning Routine” Analogy

Traditional R code evaluates from the inside out, creating a nightmare of nested parentheses. The pipe unwraps this, letting you read and write sequentially from top to bottom.

Consider a morning routine. **Without the pipe (Inside-Out):**

```
leave_house(have_breakfast(dress(shower(me))))
```

With the pipe (Left-to-Right):

```
me |>
  shower() |>
  dress() |>
  have_breakfast() |>
  leave_house()
```

The pipe simply passes “you” through a logical sequence of actions.

The Tale of Two Pipes: `%>%` vs. `|>`

You will see two different pipe symbols in the wild. They do the exact same thing:

- `%>%` (**The Magrittr Pipe**): The original pipe popularized by the Tidyverse. It requires loading packages like `dplyr` to work.
- `|>` (**The Native Pipe**): Built directly into R (Version 4.1+). It requires no extra packages and is the modern standard. We will focus on this one.

Using the Pipe with Base R

Because `|>` is built into R, you can use it to instantly clean up standard base R operations.

Here is how we take a single column, calculate the mean, and round the answer:

Without the pipe (Nested):

```
round(mean(NHANES$BMI, na.rm = TRUE), digits = 1)
```

With the pipe (Sequential):

```
NHANES$BMI |>
  mean(na.rm = TRUE) |>
  round(digits = 1)
```

How to read it: “Take the BMI column, **and then** calculate the mean, **and then** round the result to one decimal place.”

The Keyboard Shortcut

To type the pipe quickly, use the keyboard shortcut **Ctrl + Shift + M** (Mac: **Cmd + Shift + M**). To ensure RStudio inserts the modern `|>` instead of the older `%>%`, check the box for “Use native pipe operator, `|>`” under **Tools > Global Options > Code**.

V. Dataset Manipulation

```
# install.packages("tidyverse")
library(tidyverse)
```

A. Column (Variable) names

```
names(NHANES) # base R
glimpse(NHANES) # tidyverse

# To rename variable by name:

NHANES <- NHANES |>
  rename(id = ID) # rename(new_name = old_name)
head(NHANES)

#rename back
NHANES <- NHANES |>
  rename(ID = id)
```

B. Working with Missing Data

```
# examining categorical variable:
NHANES |> count(Education)
# count prints NA by default

# examining numerical variables
NHANES |> count(SleepHrsNight) # has too many values
```



```
# OR

NHANES |> count(is.na(SleepHrsNight)) # counts the missing values (TRUE)

summary(NHANES$SleepHrsNight)

mean(NHANES$SleepHrsNight, na.rm = TRUE)

# OR
# if you're planning to calculate mean after some transformation, here's the tidyverse way
# tidyverse
NHANES |>
  pull(SleepHrsNight) |>
  mean(na.rm = TRUE)
```

Note: this does not work!

```
NHANES |> mean(SleepHrsNight, na.rm = TRUE)
```

Because the pipe takes whatever is on the left and shoves it into the first argument of the function on the right. This code translates to `mean(NHANES, SleepHrsNight, na.rm = TRUE)`, which asks to calculate the mean of the whole spreadsheet, not one column.

C. Computing Variables

```
# Create New Variable
# base R (repeats dataset name, clunky)
NHANES$bmi_manual <- NHANES$Weight / (NHANES$Height / 100)^2

# tidyverse
NHANES <- NHANES |>
  mutate(
    height_meters = Height / 100,          # 1. Create temporary column
    bmi_manual = Weight / (height_meters^2), # 2. Use it for math
    height_meters = NULL                  # 3. Delete it immediately
  )

# OR just
NHANES <- NHANES |>
  mutate(
    bmi_manual = Weight / (Height / 100)^2
  )

# check
head(NHANES$BMI)
head(NHANES$bmi_manual)

# To create a binary variable, can use if_else()
NHANES <- NHANES |> mutate(college_grad = Education == "College Grad") # makes a binary var
```

```
# OR
NHANES <- NHANES |> mutate(college_grad = if_else(Education == "College Grad", 1, 0)) # makes a numerical
```

Exercise 3

- In `e2`, rename `obj1` to 'rate' and rename `obj2` to 'state'.
- Create a new variable in `e2` called `e3`, which is the square root of rate, divided by the `idnum`.

D. Recode a Continuous Variable into a New Categorical Variable

```
NHANES <- NHANES |>
  mutate(Age_Category = case_when(
    Age < 18 ~ "Minor",
    Age < 65 ~ "Adult", # We don't need to check >= 18, the line above already caught them!
    Age >= 65 ~ "Senior"
  ))

# Check the results the Tidyverse way
NHANES |>
  count(Age_Category)
```

E. Recode from categorical to categorical

```
NHANES |> count(Gender)
NHANES |> count(HealthGen)

NHANES <- NHANES |>
  mutate(
    Gender = case_match(
      Gender,
      "male" ~ "Male",
      "female" ~ "Female"
    ),
    HealthGen = case_match(
      HealthGen,
      "Vgood" ~ "Very Good",
      .default = HealthGen # Keeps any other values as they were
    )
  )
```

F. Subsets of a Data Frame

```
# Subsets by specifying the column name:
# While it is possible to subset a dataframe by specifying the exact rows and columns to keep, in practice we often use the following syntax:

df_test <- NHANES[1:5, c("Gender", "Age", "Education")]
```

```
# Keep exactly these four columns
df_test <- NHANES |>
  select(ID, Age, Gender, BMI)
View(df_test)

# Drop a column by putting a minus sign in front of it
df_test <- df_test |>
  select(-BMI) # or use select(-c(BMI, Age)) to drop more than one column
View(df_test)

# Subsets by specifying the row ###
NHANES |> filter(Age > 65) |> View()

# When filtering both rows and columns, it's best practice to filter() then select()
# 1. Datasets typically have more rows than columns, so dropping rows first reduces size
# 2. Less likely to inadvertently attempt filtering on columns that have been removed.
df_test <- NHANES |>
  filter(Age > 65, HealthGen == "Excellent") |>
  select(ID, BMI, SleepHrsNight)
```

G. Sorting Data

```
# Sort the entire dataset from youngest to oldest
NHANES |>
  arrange(Age)

# Sort from oldest to youngest using desc() for descending
NHANES |>
  arrange(desc(Age))

# Sort by Gender, and then by Age within each Gender
NHANES |>
  arrange(Gender, Age)
```

Exercise 4

Using `e2`, create a subset called `e4` which contains only the observations with a rate between 5 and 7.

Challenge: Putting it all together

Find the top 5 heaviest adult men in the dataset, but only show their ID, Age, and Weight.

Before writing code, in plain words, what are the necessary steps/transformations?

Solution:

```
NHANES |>
  filter(Age >= 18 & Gender == "male") |> # 1. Filter rows
  arrange(desc(Weight)) |>               # 2. Sort heaviest to lightest
```

```
select(ID, Age, Weight) |>  
head(5)
```

```
# 3. Keep only 3 columns  
# 4. Show the top 5
```

V. Descriptive Statistics

```
summary(NHANES)  
summary(NHANES$Age)
```

A. Continuous variable : age

```
# to get a single value:  
mean(NHANES$Age, na.rm = TRUE)  
median(NHANES$Age, na.rm = TRUE)  
sd(NHANES$Age, na.rm = TRUE)  
quantile(NHANES$Age, na.rm = TRUE)  
  
# Calculate summary statistics in a single tidy data frame using whatever functions you'd like  
NHANES |>  
  summarize(  
    Mean    = mean(Age, na.rm = TRUE),  
    Median  = median(Age, na.rm = TRUE),  
    SD      = sd(Age, na.rm = TRUE),  
    Min     = min(Age, na.rm = TRUE),  
    Q1      = quantile(Age, 0.25, na.rm = TRUE),  
    Q3      = quantile(Age, 0.75, na.rm = TRUE),  
    Max     = max(Age, na.rm = TRUE),  
    NAs     = sum(is.na(Age))  
  )  
  
library(gtsummary)  
  
NHANES |>  
  tbl_cross(  
    row = Gender,  
    col = HealthGen,  
    percent = "row",      # Calculates row percentages  
    margin = "row"        # Adds 'Total' row  
  ) |>  
  add_p()  
  
NHANES |>  
  # Step 1: Select only the variables you want in the table  
  select(Age, Gender, BMI, HealthGen) |>  
  
  # Step 2: Build the table, split into columns by Diabetes status  
  tbl_summary(by = Gender) |>  
  
  # Step 3: Add the p-values comparing the two groups  
  add_p()
```

B. Categorical variable : gender

For categorical variables, if we want simple counts and crosstabulations, we can use `count()`.

```
NHANES |>
  count(Gender)
# and
NHANES |>
  count(Gender, AgeDecade)
```

For nicely formatted crosstabulations with marginal percentages, we can use the `gtsummary` package, which not only computes values but also generates HTML that can be exported to a variety of formats.

```
# install.packages('gtsummary')
library(gtsummary)

# just counts
NHANES |>
  tbl_cross(
    row = Gender,
    col = AgeDecade
  )

# row frequencies (what % of males are in their 20s?):
NHANES |>
  tbl_cross(
    row = Gender,
    col = AgeDecade,
    percent = "row"
  )

# Column frequencies (what % of 20-somethings are male?):
NHANES |>
  tbl_cross(
    row = Gender,
    col = AgeDecade,
    percent = "column"
  )
```

C. Correlation & Plotting

```
# use select() to isolate variables to correlate
df_cor <- NHANES |> select(Age, Weight, Height)

cor(df_cor, use = "pairwise.complete.obs")
plot(df_cor) # matrix plot corresponding to selected variables
```

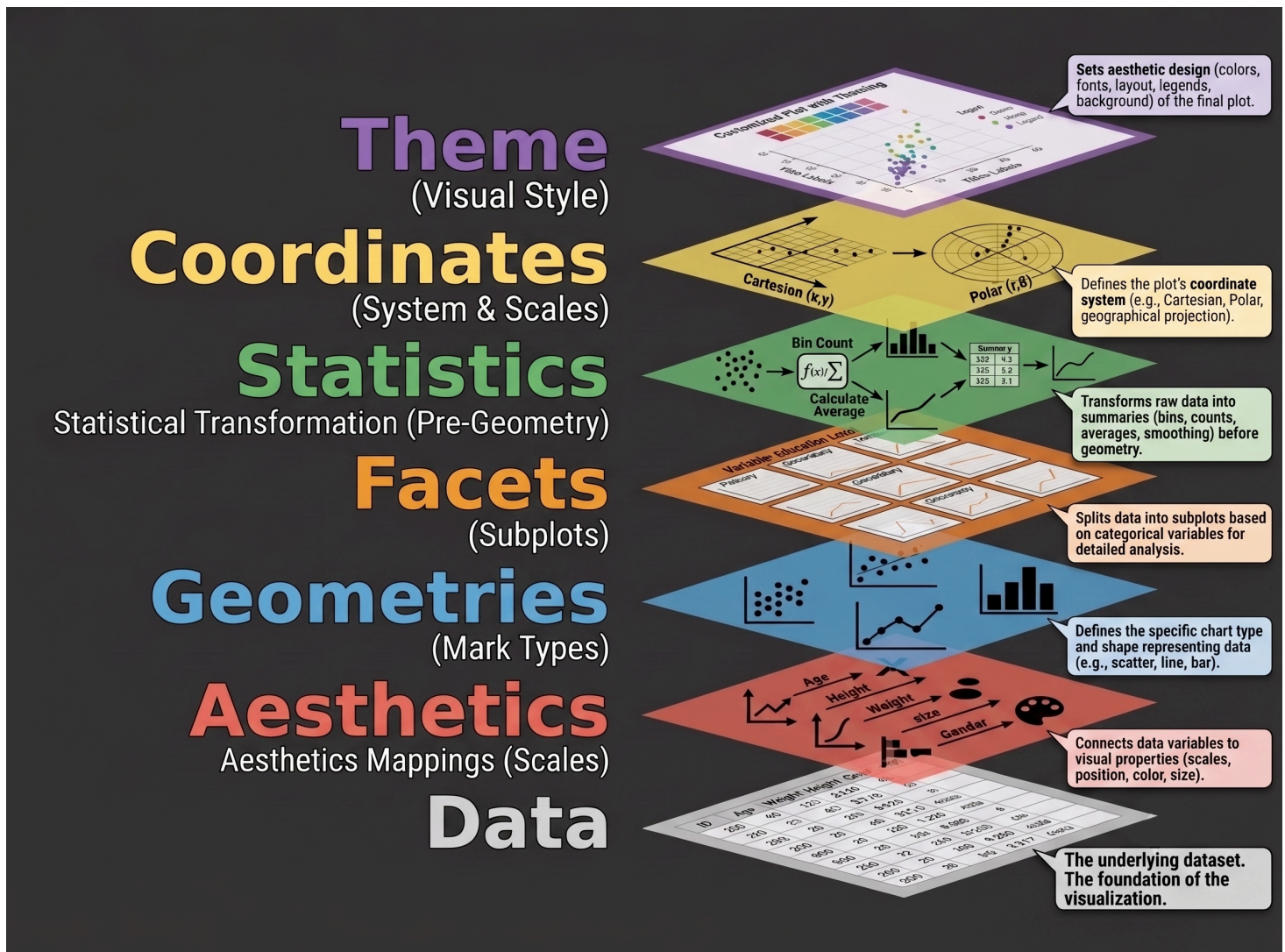
Exercise 5

Using `e2`, create an appropriate set of statistics for the `rate` variable.

VI. Graphics

Plotting with ggplot2

The highly popular `ggplot2` uses Leland Wilkinson's "Grammar of Graphics" — a system where you start with a blank canvas and stack information one layer at a time using the plus sign (+).



Example 1: The Human Lifespan (Boxplots)

We know that humans grow rapidly in childhood, plateau in adulthood, and slightly shrink in old age. Let's build a plot that proves it, layer by layer.

Layer 0: Data

Before jumping into plotting, it's a good idea to have a basic understanding of variables we are plotting.

```
summary(NHANES$AgeDecade)
summary(NHANES$Height)
```

Since there are some missing values, let's look at where they occur:

```
NHANES |>
count(
```

```
Missing_Age = is.na(AgeDecade),
Missing_Height = is.na(Height),
name = "Total_Participants"
)
```

We can use `drop_na()` to focus only on the valid values.

```
NHANES |>
drop_na(AgeDecade, Height) |>
count(
  Missing_Age = is.na(AgeDecade),
  Missing_Height = is.na(Height),
  name = "Total_Participants"
)
```

Layer 1: The Canvas

First, we pipe our cleaned data into `ggplot()` and map our axes. We put time (`AgeDecade`) on the X-axis and `Height` on the Y-axis.

```
NHANES |>
drop_na(AgeDecade, Height) |>
ggplot(aes(x = AgeDecade, y = Height))
```

Layer 2: The Geometry

We add a geometry layer to tell R to draw boxplots. This instantly reveals the biological story: the boxes climb, flatten out, and then dip.

```
NHANES |>
drop_na(AgeDecade, Height) |>
ggplot(aes(x = AgeDecade, y = Height)) +
geom_boxplot()
```

Layer 3: Theme (and other enhancements)

Finally, we add fill to make it visually engaging, a theme to clean up the gray background, and proper labels.

```
NHANES |>
drop_na(AgeDecade, Height) |>
ggplot(aes(x = AgeDecade, y = Height, fill = AgeDecade)) +
geom_boxplot(show.legend = FALSE) +
theme_minimal() +
labs(
  title = "Human Growth and Shrinkage Across the Lifespan",
  x = "Age Decade",
  y = "Height (cm)"
)
```

Example 2: Hidden Groups (Histograms & Faceting)

Sometimes, looking at the entire population hides crucial details. We can use layers and facets to solve data mysteries.

Layer 1: The Single Lump

If we just plot a histogram of adult height, the resulting bell curve looks unusually wide and a bit lumpy.

```
NHANES |>
  filter(Age >= 18) |>
  drop_na(Height) |>
  ggplot(aes(x = Height)) +
  geom_histogram()
```

Result: A single, wide mountain of data. Why not a bell curve?

Layer 2: Revealing the Hidden Groups with Color

By adding `fill = Gender` to our aesthetics, we instantly reveal the two distinct bell curves corresponding to two distinct populations hiding inside that single mountain.

```
NHANES |>
  filter(Age >= 18) |>
  drop_na(Height, Gender) |>
  ggplot(aes(x = Height, fill = Gender)) +
  geom_histogram(position = "identity", alpha = 0.6)
# position makes histogram bars show up unstacked and alpha controls transparency
```

Layer 3: Splitting the View with facet_wrap()

While overlapping colors are nice, they can sometimes be hard to read. The most powerful layer in `ggplot2` is the facet. By adding `facet_wrap(~ Gender)`, we tell R: "Take this entire chart, and split it into separate side-by-side plots by Gender."

```
NHANES |>
  filter(Age >= 18) |>
  drop_na(Height, Gender) |>
  ggplot(aes(x = Height, fill = Gender)) +
  geom_histogram(show.legend = FALSE) +
  facet_wrap(~ Gender) +
  theme_minimal() +
  labs(
    title = "Adult Height Distribution Separated by Gender",
    x = "Height (cm)",
    y = "Count"
  )
```

Exercise 6

Using `e2`, create a histogram of rate with appropriate customizations.

VII. Hypothesis Testing

Base R statistical functions like `t.test()` or `lm()` have messy outputs designed only to be read by human eyes in the console.

```
# Chi-square Test
chisq.test(NHANES$Gender, NHANES$AgeDecade)
summary(table(NHANES$Gender, NHANES$AgeDecade))

# t-test
# One sample t test
t.test(NHANES$Height, mu=150)
```

However, a package called `broom` “sweeps up” messy statistical output into clean, tidy data frames.

```
# install.packages('broom')
library(broom)
t.test(NHANES$Height, mu=150) |> tidy()

# Independent 2 group t test where y is numeric and x is a binary factor
t.test(Height ~ Gender, data = NHANES) |> tidy()

# Paired t-test
paired_t_results <- t.test(NHANES$BPSys1, NHANES$BPSys2, paired = TRUE) |> tidy()

# Because the output is a tidy dataframe, extracting the statistic is just a pull()!
paired_t_results |> pull(statistic)

# ANOVA
aov_bmi <- aov(BMI ~ Race1 + Gender, data = NHANES)
summary(aov_bmi)

# OR
aov_bmi |> tidy()

# Linear Regression Model
lm_bmi <- lm(BMI ~ Age + Gender, data = NHANES)
summary(lm_bmi)

# OR
lm_bmi |> tidy(conf.int = TRUE)
```

For better reporting and clearer summaries of regressions, use `easystats`.

```
# install.packages("easystats")
library(easystats)

# Generate the automated text report
report(lm_bmi)

model_parameters(lm_bmi)
```

```
check_model(lm_bmi)
```

Exercise Solutions

Of course, there is always more than 1 solution; below are just examples.

Exercise 1

Create a vector called `e1` which is a sequence of 1 to 10, repeating 5 times.

```
e1 <- rep(1:10, times = 5)

v <- c(1:10)
e1 <- rep(v, times = 5)
```

Exercise 2

Create a data frame called `e2`, which adds a third variable (`idnum`, see below) to `df1`.

```
idnum <- seq(from = 1, to = 10)
e2 <- data.frame(idnum, df1)
```

OR

```
e2 <- df1 |> mutate(idnum = row_number()) |> relocate(idnum) # Moves idnum to the first column
```

Exercise 3

- In `e2`, rename `vec1` 'rate' and rename `vec2` 'state'.
- Create a new variable in `e2` called `e3`, which is the sqrt of rate, divided by the `idnum`.

```
names(e2)[2:3] <- c('rate', 'state')
e2$e3 <- sqrt(e2$rate) / e2$idnum

names(e2)[names(e2) %in% c("vec1","vec2")] <- c('rate', 'state')
```

OR

```
e2 <- e2 |> rename( rate = vec1, state = vec2 ) |> mutate(e3 = sqrt(rate) / idnum)
```

Exercise 4

Using `e2`, create a subset called `e4` which contains only the observations with a rate between 5 and 7.

```
e4 <- subset(e2, rate > 5 & rate < 7)
```

OR

```
e4 <- e2 |> filter(rate > 5, rate < 7)
```

Exercise 5

Using `e2`, create an appropriate set of statistics for the rate variable.

```
summary(e2$rate)
mean(e2$rate)
max(e2$rate)
quantile(e2$rate)
```

OR

```
e2 |>
  summarize(
    mean_rate = mean(rate, na.rm = TRUE),
    median_rate = median(rate, na.rm = TRUE),
    min_rate = min(rate, na.rm = TRUE),
    max_rate = max(rate, na.rm = TRUE),
    sd_rate = sd(rate, na.rm = TRUE)
  )
```

Exercise 6

Using `e2`, create a histogram of rate with appropriate customizations.

```
hist(e2$rate, xlab = "Rate", main = "Histogram of Rate", col = "green")
```

OR

```
e2 |>
  ggplot(aes(x = rate)) +
  geom_histogram(fill = "green", color = "black", bins = 10) +
  theme_minimal() +
  labs(
    title = "Histogram of Rate",
    x = "Rate",
    y = "Count"
  )
```